

StyleGaussian: Instant 3D Style Transfer with Gaussian Splatting

Kunhao Liu
Nanyang Technological University
Singapore
kunhao001@e.ntu.edu.sg

Fangneng Zhan
Max Planck Institute for Informatics
Germany
fzhan@mpi-inf.mpg.de

Muyu Xu
Nanyang Technological University
Singapore
muyu001@e.ntu.edu.sg

Christian Theobalt
Max Planck Institute for Informatics
Germany
theobalt@mpi-inf.mpg.de

Ling Shao
UCAS-Terminus AI Lab, UCAS
China
ling.shao@ieee.org

Shijian Lu
Nanyang Technological University
Singapore
shijian.lu@ntu.edu.sg

Abstract

We introduce StyleGaussian, a novel 3D style transfer technique that allows instant transfer of any image’s style to a 3D radiance field at 10 frames per second (fps). Leveraging 3D Gaussian Splatting (3DGS), StyleGaussian achieves style transfer without compromising its real-time rendering ability and multi-view consistency. It achieves instant style transfer with three steps: embedding, transfer, and decoding. Initially, 2D VGG scene features are embedded into reconstructed 3D Gaussians. Next, the embedded features are transformed according to a reference style image. Finally, the transformed features are decoded into the stylized RGB. StyleGaussian has two novel designs. The first is an efficient feature rendering strategy that first renders low-dimensional features and then maps them into high-dimensional features while embedding VGG features. It cuts the memory consumption significantly and enables 3DGS to render the high-dimensional memory-intensive features. The second is a K-nearest-neighbor-based 3D CNN. Working as the decoder for the stylized features, it eliminates the 2D CNN operations that compromise strict multi-view consistency. Extensive experiments show that StyleGaussian achieves instant 3D stylization with superior stylization quality while preserving real-time rendering and strict multi-view consistency. Project page: <https://kunhao-liu.github.io/StyleGaussian/>

CCS Concepts

• Computing methodologies → Non-photorealistic rendering; Computer vision.

Keywords

3D Gaussian Splatting, 3D Style Transfer, Radiance Fields

ACM Reference Format:

Kunhao Liu, Fangneng Zhan, Muyu Xu, Christian Theobalt, Ling Shao, and Shijian Lu. 2024. StyleGaussian: Instant 3D Style Transfer with Gaussian Splatting. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications ’24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3698002>



This work is licensed under a Creative Commons Attribution International 4.0 License.

SA Technical Communications ’24, December 03–06, 2024, Tokyo, Japan
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1140-4/24/12
<https://doi.org/10.1145/3681758.3698002>

1 Introduction

Remarkable advancements in radiance fields [Mildenhall et al. 2021] have revolutionized 3D reconstruction, enabling immersive 3D exploration. A radiance field maps 3D coordinates to color and density values using implicit MLPs, explicit voxels, or a combination of both. However, editing these fields is more complex than editing meshes or point clouds. Studies have explored radiance field editing via images [Liu et al. 2023; Zhang et al. 2022], text, or other inputs, but these often require time-intensive test-time optimization [Zhang et al. 2022]. Feed-forward networks offer some speed-up but can compromise multi-view consistency [Liu et al. 2023].

Instant interaction and feedback are crucial for a seamless 3D user experience. 3D Gaussian Splatting (3DGS) [Kerbl et al. 2023] offers rapid rendering, making it possible to achieve instant 3D style transfer. We introduce StyleGaussian, a 3D style transfer pipeline that enables instant 3D style transfer while preserving multi-view consistency. The contributions of this work are threefold: *First*, StyleGaussian enables instant style transfer (10 fps) with real-time rendering and strict multi-view consistency. *Second*, we design an efficient feature rendering strategy for embedding high-dimensional features into 3D Gaussians. *Third*, we develop a KNN-based 3D CNN that decodes stylized features to RGB without impairing multi-view consistency.

2 StyleGaussian

We present StyleGaussian, a 3D style transfer method that can transfer the style of any image to reconstructed 3D Gaussians instantly. Given the reconstructed 3D Gaussians $\mathbb{G} = \{g_p = (\mu_p, \Sigma_p, \sigma_p, c_p)\}_p^P$ of a scene, and a style image I^s , our goal is to stylize the color of each Gaussian such that the stylized 3D Gaussians $\mathbb{G}^s = \{g_p^s\}_p^P$ capture the style of I^s while keeping their original content structure. For each stylized Gaussian g_p^s , we fix its geometry and only transform the color: $g_p^s : c_p \rightarrow c_p^s$. The stylized 3D Gaussians $\mathbb{G}^s$ have the same rendering process as the original 3D Gaussians $\mathbb{G}$, thus maintaining real-time rendering and strict multi-view consistency.

We denote the style transfer process as $f : f(\mathbb{G}, I^s) = \mathbb{G}^s$. Inspired by the zero-shot style transfer literature [Huang and Belongie 2017; Liu et al. 2023], we decompose f into three steps, as shown in fig. 1: **1) embedding** e , which embeds the 2D VGG image features into $\mathbb{G}$:

$$e(\mathbb{G}) = \mathbb{G}^e = \{g_p^e = (\mu_p, \Sigma_p, \sigma_p, f_p)\}_p^P, \quad (1)$$

where each embedded Gaussian g_p^e has its VGG feature f_p ; **2) transfer** t , which transforms the features of the embedded 3D



Figure 1: Overview of StyleGaussian. Given a reconstructed 3D Gaussians $\mathbb{G}$, we first embed the VGG features to the 3D Gaussians (e). Then, given a style image, we transform the features of the embedded Gaussians $\mathbb{G}^e$ to obtain $\mathbb{G}^t$, where the features are infused with the style information (t). Lastly, we decode the transformed features of $\mathbb{G}^t$ into RGB to produce the final stylized 3D Gaussians $\mathbb{G}^s$ (d). We design an efficient feature rendering strategy in e that enables rendering high-dimensional VGG features while learning to embed them into $\mathbb{G}$. We also develop a KNN-based 3D CNN as the decoder in d.

Gaussians $\mathbb{G}^e$ according to the input style image I^s :

$$t(\mathbb{G}^e, I^s) = \mathbb{G}^t = \{g_p^t = (\mu_p, \Sigma_p, \sigma_p, f_p^t)\}_p^P, \quad (2)$$

where the transformed feature of each Gaussian f_p^t has been infused with the style information of I^s ; **3) decoding d**, which decodes the transformed features of 3D Gaussians back to RGB:

$$d(\mathbb{G}^t) = \mathbb{G}^s = \{g_p^s = (\mu_p, \Sigma_p, \sigma_p, c_p^s)\}_p^P. \quad (3)$$

Note that the geometry properties $\mu_p, \Sigma_p, \sigma_p$ of each Gaussian g_p are fixed during the whole style transfer process, only the feature of each Gaussian changes: $c_p \xrightarrow{e} f_p \xrightarrow{t} f_p^t \xrightarrow{d} c_p^s$. In summary, the whole style transfer process can be described as $\{c_p \xrightarrow{f} c_p^s, I^s\}_p^P, f = e \circ t \circ d$. We will elaborate on the feature embedding process e in section 2.1, the style transfer process t in section 2.2, and the RGB decoding process d in section 2.3.

2.1 Feature Embedding

The VGG features of a 2D image are extracted from the intermediate layers of a pre-trained VGG network. To embed these features into a radiance field, we distill features from multi-view 2D VGG maps. Since Gaussian Splatting is scene-specific, we embed VGG features for 3D Gaussians after reconstruction and keep them fixed during stylization.

Given the reconstructed 3D Gaussians of a scene, each Gaussian g_p is assigned a learnable feature parameter $f_p \in \mathbb{R}^D$. We render the feature of each pixel $F \in \mathbb{R}^D$ similarly to rendering RGB:

$$F = \sum_{i \in N} f_i w_i, \quad w_i = \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (4)$$

where w_i is the blending weight of each Gaussian. We then optimize $\{f_p\}_p^P$ using the L1 loss objective between the rendered features and the ground truth VGG features: $\{f_p\}_p^P = \argmin \sum |F - F_{gt}|$.

Rendering high-dimensional VGG features directly through Gaussian Splatting is computationally and memory intensive, especially with channel sizes $D = 256$ in ReLU3_1 and $D = 512$ in ReLU4_1. For example, with $D = 256$, the renderer exceeds GPU memory limits. To address this, we propose rendering low-dimensional features $F' \in \mathbb{R}^{D'}$, with $D' = 32$, before mapping them to high-dimensional features F . Each Gaussian's learnable feature parameter is now

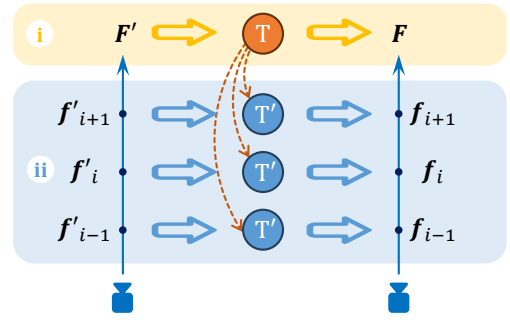


Figure 2: Efficient feature rendering. We first render the low-dimensional features F' and then map them to high-dimensional features F . Part i corresponds to eq. (5), where T denotes the affine transformation applied to F' . Part ii corresponds to eq. (6), where T' denotes the affine transformation applied to f'_i .

$f_p' \in \mathbb{R}^{D'}$, and the low-dimensional feature of each pixel is rendered as $F' = \sum_{i \in N} f'_i w_i$.

The challenge is mapping F' to F while obtaining the high-dimensional VGG feature f_p of each Gaussian. We design this mapping as an affine transformation $T(\cdot)$:

$$F = T(F') = AF' + \sum_{i \in N} b w_i, \quad (5)$$

where $A \in \mathbb{R}^{D \times D'}$ is the learnable linear transformation, $b \in \mathbb{R}^D$ is the learnable bias, and w_i is the blending weight in eq. (4). The optimization objective is now: $A, b, \{f'_p\}_p^P = \argmin \sum |F - F_{gt}|$.

Since our goal is to obtain f_p and we now only possess the optimized low-dimensional features f'_p , we proceed to derive the relationship between f'_p and f_p . Note that eq. (5) can be written as:

$$F = A \sum_{i \in N} f'_i w_i + \sum_{i \in N} b w_i = \sum_{i \in N} (A f'_i + b) w_i = \sum_{i \in N} T'(f'_i) w_i, \quad (6)$$

where $T'(\cdot)$ denotes the affine transformation applied to f'_i . With eq. (4) and eq. (6), it is obvious that $f_p = T'(f'_p) = A f'_p + b$. The visualization of the feature rendering process is in fig. 2. We now obtain the VGG features $\{f_p\}_p^P$ of the 3D Gaussians. The

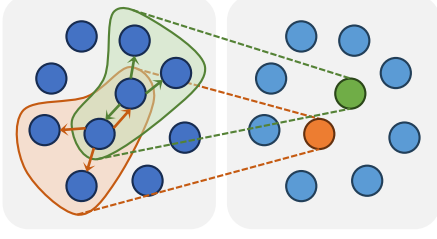


Figure 3: Illustration of the KNN-based convolution. We treat each Gaussian’s KNN as the sliding window.

VGG features of 3D Gaussians are pre-optimized and fixed during stylization, thus the embedding process does not affect the speed of style transfer.

2.2 Style Transfer

After obtaining the 3D Gaussians $\mathbb{G}^e$ embedded with VGG features, we transform these features according to an arbitrary input style image I^s . For real-time style transfer, we use the efficient, parameter-free zero-shot style transfer algorithm AdaIN [Huang and Belongie 2017]. AdaIN performs style transfer by aligning the channel-wise mean and variance of content features with those of the style features. The transformed feature $f_p^t \in \mathbb{R}^D$ of a Gaussian is given by:

$$f_p^t = \sigma(F^s) \left(\frac{f_p - \mu(\{f_p\}_p^P)}{\sigma(\{f_p\}_p^P)} \right) + \mu(F^s), \quad (7)$$

where $\mu(\cdot) \in \mathbb{R}^D$, $\sigma(\cdot) \in \mathbb{R}^D$ denote the channel-wise mean and standard deviation, F^s denotes the VGG features of the input style image I^s . We thus obtain the transformed 3D Gaussians $\mathbb{G}^t$ where each Gaussian’s feature f_p^t has been infused with the style information.

2.3 RGB Decoding

After obtaining the transformed 3D Gaussians $\mathbb{G}^t$, we decode the features directly into RGB in 3D to obtain the final stylized 3D Gaussians $\mathbb{G}^s$. This method ensures real-time rendering and strict multi-view consistency by maintaining the 3D Gaussian rendering process. Unlike previous methods that render feature maps before decoding, our approach decodes features directly. While a per-point MLP could be used, research shows a large receptive field is needed for accurate RGB decoding, which is why previous studies have preferred 2D CNNs over MLPs [Huang et al. 2021; Liu et al. 2023].

We design a simple and effective 3D CNN as the decoder for decoding the transformed features to the stylized RGB. The decoder processes each Gaussian’s K-nearest neighbors (KNN) within a sliding window and is composed of multiple layers, as shown in fig. 3. For each Gaussian g_p^t , we denote its KNN as $\{g_k^t\}_k^K$, noting that the center Gaussian itself is also included in its KNN. Therefore, the input features for the first layer of the sliding window centered at g_p^t are $\{f_k^t\}_k^K$. For each layer, we refer to the input features of a sliding window in this layer as $\{f_k^{in} \in \mathbb{R}^{D_{in}}\}_k^K$. The output of the sliding window, assigned to the center Gaussian g_p^t , is denoted as $f_p^{out} \in \mathbb{R}^{D_{out}}$. The parameters of the convolution operations in a

Table 1: Quantitative results. We evaluate the performance of StyleGaussian against the state-of-the-art in terms of consistency and performance regarding transfer time and rendering time.

Methods	Short-range Consistency		Long-range Consistency		Transfer Time	Rendering Time
	<i>LPIPS</i>	<i>RMSE</i>	<i>LPIPS</i>	<i>RMSE</i>	<i>Seconds</i>	<i>Seconds</i>
HyperNet	0.036	0.043	0.076	0.078	1.908	32.373
StyleRF	0.050	0.045	0.123	0.098	3.403	10.084
StyleGaussian	0.026	0.031	0.072	0.073	0.105	0.005

layer include weights and bias terms, with the weights denoted as $\{W_k \in \mathbb{R}^{D_{out} \times D_{in}}\}_k^K$ and the bias as $B \in \mathbb{R}^{D_{out}}$. Consequently, the convolution operation in the sliding window centered at g_p^t can be expressed as:

$$f_p^{out} = \phi \left(\sum_{k=1}^K W_k f_k^{in} + B \right), \quad (8)$$

where ϕ denotes the Sigmoid function. This convolution can be efficiently implemented through matrix multiplication. We stack the features of all P Gaussians’ KNN to form the input feature tensor $F_{in} \in \mathbb{R}^{P \times K \times D_{in}}$, which is subsequently reshaped to a matrix $\tilde{F}_{in} \in \mathbb{R}^{P \times (K \times D_{in})}$. Similarly, we also stack all K weights to form $W \in \mathbb{R}^{D_{out} \times D_{in} \times K}$, which is then reshaped to another matrix $\tilde{W} \in \mathbb{R}^{D_{out} \times (D_{in} \times K)}$. Hence, we can then perform the convolution of eq. (8) as:

$$F_{out} = \phi \left(\tilde{F}_{in} \tilde{W}^T + \hat{B} \right) \in \mathbb{R}^{P \times D_{out}}, \quad (9)$$

where T denotes transpose, $\hat{B}$ indicates that B is broadcasted to $\mathbb{R}^{P \times D_{out}}$. The operations described in eq. (8) and eq. (9) are equivalent, but eq. (9) is easier to implement to be executed parallelly on GPU. Consequently, we obtain the stylized 3D Gaussians $\mathbb{G}^s$, which are capable of being rendered in real-time while maintaining strict multi-view consistency. We use the same training loss as [Huang and Belongie 2017] in the stylization training.

3 Experiments

We evaluate StyleGaussian on two real-world scene datasets: the Tanks and Temples dataset [Knapitsch et al. 2017] and the Mip-NeRF 360 dataset [Barron et al. 2022]. Both datasets contain real-world unbounded captures with complex geometries and intricate details. We also experiment with a diverse set of style images to test our method’s ability to handle a wide range of style exemplars. We compare our method with two state-of-the-art zero-shot radiance field style transfer methods HyperNet [Chiang et al. 2022] and StyleRF [Liu et al. 2023].

We provide visualizations in fig. 4 showing that StyleGaussian effectively applies various styles to 3D scenes, faithfully transferring color tone, distribution, and stroke sizes in a zero-shot manner. Quantitative comparisons on multi-view consistency, transfer, and rendering time are shown in table 1. Using a 3D CNN for decoding, StyleGaussian ensures strict multi-view consistency, outperforming baselines in stylization consistency. It also achieves transfer time speed-ups of 18x over HyperNet and 32x over StyleRF, and rendering speed-ups of 6000x and 2000x, respectively.



Figure 4: Stylization visualizations. All styles are transferred instantly in a zero-shot manner.

Acknowledgments

This project is funded by the Ministry of Education Singapore, under the Tier-1 project scheme with project number RT18/22.

References

- Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. 2022. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5470–5479.
- Pei-Ze Chiang, Meng-Shiun Tsai, Hung-Yu Tseng, Wei-Sheng Lai, and Wei-Chen Chiu. 2022. Stylizing 3D scene via implicit representation and HyperNetwork. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 1475–1484.
- Hsin-Ping Huang, Hung-Yu Tseng, Saurabh Saini, Maneesh Singh, and Ming-Hsuan Yang. 2021. Learning to stylize novel views. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 13869–13878.
- Xun Huang and Serge Belongie. 2017. Arbitrary style transfer in real-time with adaptive instance normalization. In *Proceedings of the IEEE international conference on computer vision*. 1501–1510.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (2023).
- Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. 2017. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics (ToG)* 36, 4 (2017), 1–13.
- Kunhao Liu, Fangneng Zhan, Yiwen Chen, Jiahui Zhang, Yingchen Yu, Abdulmotaleb El Saddik, Shijian Lu, and Eric P Xing. 2023. StyleRF: Zero-shot 3D Style Transfer of Neural Radiance Fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8338–8348.
- Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. 2021. Nerf: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* 65, 1 (2021), 99–106.
- Kai Zhang, Nick Kolkin, Sai Bi, Fajun Luan, Zexiang Xu, Eli Shechtman, and Noah Snavely. 2022. ARF: Artistic Radiance Fields. In *European Conference on Computer Vision*. Springer, 717–733.